JDBC in the Course Enrollment program

The JDBC code supports the same database operations in both the console and Swing versions. The examples below follow the console version we wrote.

Connecting to the database

The `connect()` method opens a connection:

```
private static Connection connect() throws SQLException {
    return DriverManager.getConnection(
            DB_URL, DB_USER, DB_PASSWORD);
}
```

`DB_URL` identifies the database server and database name. `DB_USER` and `DB_PASSWORD` provide the login credentials. `DriverManager.getConnection()` returns a `Connection` object that the program uses to send SQL statements to MySQL.

Our program opens a connection for each insert or query and closes it when that operation finishes.

Preparing SQL statements

The program uses `PreparedStatement` to execute SQL. For example, adding a student uses:

```
String sql =
        "INSERT INTO Student "
        + "(StudentID, FirstName, LastName, ClassLevel, Email) "
        + "VALUES (?, ?, ?, ?, ?)";
```

Each `?` is a placeholder for a value supplied separately by the program. The values correspond, in order, to the five listed columns.

This approach keeps user input separate from the SQL instructions, helps prevent SQL injection, and handles values such as names containing apostrophes.

Binding parameter values

The console program's `bindParameters()` method supplies values for the placeholders. It selects a JDBC setter based on the Java value's type:

```
statement.setInt(position, (Integer) value);
statement.setDate(position, (Date) value);
statement.setString(position, value.toString());
```

For example, a student ID uses `setInt()`, an enrollment date uses `setDate()`, and a course title uses `setString()`.

JDBC parameter positions begin at **1**, so the method converts the array index with:

```
int position = i + 1;
```

When an optional text field is blank, the program supplies SQL `NULL`:

```
statement.setNull(position, Types.VARCHAR);
```

This represents a missing value, such as an unassigned grade.

Inserting records

The `executeInsert()` helper performs the inserts:

```
try (Connection connection = connect();
     PreparedStatement statement =
             connection.prepareStatement(sql)) {

    bindParameters(statement, parameters);

    int rows = statement.executeUpdate();
    System.out.println(rows + " record(s) inserted.");
}
```

`prepareStatement()` creates the statement, `bindParameters()` supplies its values, and `executeUpdate()` sends it to MySQL. The returned integer indicates how many rows were affected.

The program reuses this helper to insert records into `Student`, `StudentPhone`, `Course`, and `Enrollment`.

MySQL enforces the database constraints. For example, inserting an enrollment requires an existing student and course because the enrollment record references those tables.

Retrieving records

The `showQueryResults()` helper executes `SELECT` statements:

```
try (ResultSet results = statement.executeQuery()) {
    while (results.next()) {
        // Read and display the current row.
    }
}
```

`executeQuery()` returns a `ResultSet`. Its cursor initially sits before the first row. Each call to `next()` advances to another row and returns `false` when no rows remain.

The program uses this process to list students, list student phone numbers, and find courses for a particular student.

Searching by student ID

The course search joins the `Enrollment` and `Course` tables:

```
SELECT c.CourseTitle AS CourseTitle,
       e.Semester,
       e.Grade
FROM Enrollment AS e
JOIN Course AS c
    ON e.CourseCode = c.CourseCode
WHERE e.StudentID = ?
ORDER BY e.EnrollmentDate, c.CourseTitle
```

`Enrollment` supplies the semester and grade, while `Course` supplies the course title. The placeholder limits the results to the student ID entered by the user.

The Java method passes that ID to the query helper:

```
showQueryResults(
        "Courses for Student " + studentID, sql, studentID);
```

The helper accepts `Object... parameters`, meaning zero or more additional values. This allows the same method to run both queries without parameters and searches that require them.

Reading column information

The program uses `ResultSetMetaData` so the query helper can display different sets of columns:

```
ResultSetMetaData metadata = results.getMetaData();
int columnCount = metadata.getColumnCount();
```

For each column, it retrieves the label and value:

```
String label = metadata.getColumnLabel(column);
Object value = results.getObject(column);
```

This makes the helper reusable: it can display student details for one query and course titles, semesters, and grades for another. In the Swing version, this information supplies the column headings and rows for the results table.

Closing resources and handling errors

The `try (...)` blocks use **try-with-resources**, which automatically closes the `Connection`, `PreparedStatement`, and `ResultSet`, including when an error occurs.

Database operations can throw `SQLException`. The console program catches it and displays the database error message:

```
catch (SQLException exception) {
    System.out.println(
            "Database error: " + exception.getMessage());
}
```

Possible causes include incorrect login credentials, an unavailable database server, a duplicate primary key, or a foreign key violation.

Together, these JDBC components let the Course Enrollment program turn user requests into database operations and present the stored information back to the user.